

PRÁCTICA 2

PRÁCTICA 2: TÉCNICAS DE BÚSQUEDA DE POBLACIONES PARA
EL PROBLEMA DE ASIGNACIÓN CON RESTRICCIONES (PAR)



UNIVERSIDAD DE GRANADA

Metaheurísticas

Grupo A2: Viernes 15:30 - Curso 2025-2026

Jorge García Gámiz

DNI: 77449859G

e-mail: jorgegarciag@correo.ugr.es

Índice

1. Descripción del Problema	2
2. Aplicación de los algoritmos al problema	3
3. Estructura de los algoritmos	7
3.1. Algoritmos Genéticos Generacionales (AGG)	7
3.2. Algoritmos Genéticos Estacionarios (AGE)	8
3.3. Algoritmos Meméticos (AM)	9
4. Estructura del código	11
4.1. Manual de compilación	11
4.2. Manual de ejecución	12
5. Experimentación y Análisis de Resultados	13
5.1. Descripción de los casos de prueba	13
5.2. Parámetros de ejecución	13
5.3. Resultados obtenidos	14
5.4. Análisis de resultados	16
5.4.1. Comparación general entre algoritmos	17
5.4.2. Impacto de las restricciones en la función objetivo	17
5.4.3. Esfuerzo computacional y convergencia	18
5.4.4. Comparación entre conjuntos de datos y dimensionalidad	18
5.4.5. Análisis de la variabilidad de los resultados	18
5.4.6. Eficiencia computacional y tiempos de ejecución	24
5.4.7. Conclusiones generales del análisis	25
6. Contraste de Hipótesis mediante el Test de Wilcoxon	26
7. Referencias y recursos utilizados	28
8. Anexo - Falta de espacio	29

1. Descripción del Problema

El Problema de Asignación con Restricciones (PAR) es una generalización del problema de agrupamiento clásico que busca particionar un conjunto de n instancias en k clusters disjuntos, donde cada instancia $\vec{x}_i \in \mathbb{R}^d$ tiene d características numéricas.

Además, se dispone de una matriz de restricciones $R \in M_{n \times n}$, donde:

- $R_{ij} = 1$ es Must-Link (ML): i y j deben pertenecer al mismo cluster
- $R_{ij} = -1$ es Cannot-Link (CL): i y j deben estar en distintos clusters
- $R_{ij} = 0$ indica ausencia de restricción.

Una solución se representa como un vector de asignación $s = (s_1, \dots, s_n)$ con $s_i \in \{0, \dots, k-1\}$ asigna cada instancia a un cluster.

Función Objetivo

El objetivo es obtener una partición solución que permita minimizar la función fitness definida como:

$$fitness(s) = desviacion(s) + \lambda \cdot infeasibility(s), \quad (1)$$

donde el parámetro $\lambda = \frac{\max_{i,j} \|\vec{x}_i - \vec{x}_j\|}{R} \in [0, 1[$ equilibra ambos términos (R es el número de restricciones).

Desviación

La calidad del clustering se mide mediante la desviación general. Primero se calculan los centroides. Para cada cluster C_i , con $i \in \{0, \dots, k-1\}$, su centroide es:

$$\vec{\mu}_i = \frac{1}{|C_i|} \sum_{\vec{x}_j \in C_i} \vec{x}_j$$

Luego la distancia media intra-cluster, que es la media de las distancias de las instancias que lo conforman a su centroide:

$$dist_{intra-cluster}(C_i) = \frac{1}{|C_i|} \sum_{\vec{x}_j \in C_i} \|\vec{x}_j - \vec{\mu}_i\|$$

Definimos la desviación general de la partición en clusters $C = \{C_1, \dots, C_k\}$ como la media de las desviaciones intra-cluster de cada cluster:

$$desviacion(C) = \frac{1}{k} \sum_{i=1}^k dist_{intra-cluster}(C_i)$$

Infeasibility

La “infeasibility” cuenta el número de restricciones violadas:

$$infeasibility = \sum_{(i,j) \in ML} [s_i \neq s_j] + \sum_{(i,j) \in CL} [s_i = s_j]$$

2. Aplicación de los algoritmos al problema

Hemos implementado siete algoritmos para esta nueva resolución del problema PAR: dos variantes del Algoritmo Genético Generacional (AGG), otras dos del Algoritmo Genético Estacionario (AGE), y tres variantes de Algoritmos Meméticos (AM). La clase `GeneticAlgorithm` hereda directamente de la clase base `MH`, implementando el método `optimize()` de la interfaz, lo que permite integrarla en el marco de algoritmos existente sin modificarel diseño original.

Los algoritmos implementados comparten una misma estructura básica de representación, inicialización y evaluación. En esta sección se describen dichos elementos comunes, así como los operadores genéticos fundamentales.

Esquema de representación y Función Objetivo

Por falta de espacio, dado que éstos son idénticos a los utilizados en la práctica anterior, se omite su descripción.

Clase `GeneticAlgorithm`

La clase `GeneticAlgorithm` centraliza la lógica de las poblaciones y hereda de `MH`. Su comportamiento se configura dinámicamente en el constructor a través de dos enumerados: `GAType`, que define la arquitectura general del algoritmo (Generacional, Estacionario, o una de las tres variantes Meméticas), y `CrossoverType`, que determina si se empleará un cruce uniforme o por segmento fijo.

El constructor también establece parámetros clave como el tamaño de la población (por defecto 50), el límite de evaluaciones (100.000), y las probabilidades de cruce (0,8) y mutación ($0,1/N$, donde N es el número de genes).

A nivel de eficiencia, un detalle de diseño destacable es la inclusión de vectores paralelos (`pop_fitness` y `off_fitness`) que almacenan las evaluaciones de las soluciones. Esto evita recalcular la costosa función objetivo de soluciones que no han sido alteradas durante el ciclo evolutivo.

A continuación, se detallan los operadores genéticos diseñados:

Selección por Torneo ($k = 3$)

El método `tournamentSelection()` elige un individuo de la población actual para ser padre en la generación siguiente. En esta implementación el tamaño del torneo es fijo e igual a 3. Su funcionamiento es el siguiente:

1. Se selecciona aleatoriamente un primer individuo como “mejor” candidato inicial.
2. Se extraen otros dos individuos al azar (con posible repetición).
3. Para cada participante, si su valor de *fitness* es menor (mejor) que el del mejor actual, se convierte en el nuevo mejor.

Al final se devuelve el índice del individuo ganador del torneo. Dado que se minimiza la función objetivo, un *fitness* más bajo indica una mejor calidad.

```
function TOURNAMENTSELECTION
    mejor  $\leftarrow$  entero aleatorio entre 0 y pop_size - 1
    for i  $\leftarrow$  1 hasta 2 do
        contestant  $\leftarrow$  entero aleatorio entre 0 y pop_size - 1
        if pop_fitness[contendiente] < pop_fitness[mejor] then
            mejor  $\leftarrow$  contendiente
        end if
    end for
    return mejor
end function
```

Operadores de Cruce y Mecanismo de Reparación

Dado que la recombinación de dos soluciones en el problema PAR puede generar asignaciones infactibles (clústeres vacíos o violaciones de restricciones), el algoritmo delega el cruce en dos métodos específicos según el `CrossoverType`, ambos apoyados por un mecanismo de reparación (`repairSolution`).

Para el **Cruce Uniforme**, las primeras $\frac{n}{2}$ posiciones del cromosoma (en ese orden) se heredan del padre respectivo (hijo1 recibe del padre1, hijo2 del padre2). Las restantes $\frac{n}{2}$ posiciones se intercambian (hijo1 toma el valor del padre2, y viceversa). Este operador promueve una alta diversificación en el espacio de búsqueda.

```
function UNIFORMCROSSOVER(padre1, padre2)
    n  $\leftarrow$  longitud del cromosoma
    indices  $\leftarrow$  [0, 1, ..., n - 1]
    indices  $\leftarrow$  SHUFFLE(indices)
    half  $\leftarrow$  n/2
    hijo1  $\leftarrow$  COPIA(padre1)
    hijo2  $\leftarrow$  COPIA(padre2)
     $\triangleright$  Intercambiar la segunda mitad de los genes (según orden aleatorio)
    for i  $\leftarrow$  half hasta n - 1 do
        idx  $\leftarrow$  indices[i]
        hijo1[idx]  $\leftarrow$  padre2[idx]
        hijo2[idx]  $\leftarrow$  padre1[idx]
    end for
    padre1  $\leftarrow$  hijo1
    padre2  $\leftarrow$  hijo2
    REPAIRSOLUTION(padre1)
    REPAIRSOLUTION(padre2)
end function
```

Para el **Cruce por Segmento Fijo** se selecciona un punto de inicio `pos` y una longitud `fn` (máximo un tercio del cromosoma). El primer segmento se hereda intacto de un padre, y el resto se completa con los valores del otro, lo que ayuda a preservar bloques de asignaciones contiguas que ya funcionaban bien.

```

function FIXEDSEGMENTCROSSOVER(padre1, padre2)
   $n \leftarrow$  longitud del cromosoma
   $pos \leftarrow$  ALEATORIO(0,  $n - 1$ ) ▷ punto de inicio del segmento
   $fn \leftarrow$  ALEATORIO(1,  $\text{máx}(1, n/3)$ ) ▷ longitud del segmento
   $final\_pos \leftarrow pos + fn$ 
   $hijo1 \leftarrow$  COPIA(padre1)
   $hijo2 \leftarrow$  COPIA(padre2)
  for  $i \leftarrow 0$  hasta  $n - 1$  do
     $en\_segmento \leftarrow$  falso
    ▷ El segmento puede dar la vuelta (wrap-around)
    if  $final\_pos < n$  then
      if  $i \geq pos$  y  $i \leq final\_pos$  then
         $en\_segmento \leftarrow$  verdadero
      end if
    else
       $final\_rem \leftarrow final\_pos \bmod n$ 
      if  $i \geq pos$  o  $i \leq final\_rem$  then
         $en\_segmento \leftarrow$  verdadero
      end if
    end if
    if  $en\_segmento$  then ▷ Dentro del segmento: intercambio obligatorio
       $hijo1[i] \leftarrow padre2[i]$ 
       $hijo2[i] \leftarrow padre1[i]$ 
    else ▷ Fuera del segmento: intercambio con probabilidad 0.5
      if random() > 0,5 then
         $hijo1[i] \leftarrow padre2[i]$ 
         $hijo2[i] \leftarrow padre1[i]$ 
      end if
    end if
  end for
   $padre1 \leftarrow hijo1$ 
   $padre2 \leftarrow hijo2$ 
  REPAIRSOLUTION( $padre1$ )
  REPAIRSOLUTION( $padre2$ )
end function

```

Ambos operadores trabajan sobre la descendencia ya generada (normalmente tras la selección), emparejando individuos consecutivos: (0,1), (2,3), etc. La probabilidad de aplicar cruce se controla con el parámetro *p-cross*

Por falta de espacio, los detalles del método de reparación de soluciones (así como su pseudocódigo) se encuentran en el Anexo (sección 8).

Operador de Mutación

La mutación (*mutate*) se aplica sobre la población descendiente en función de la probabilidad *p-mut*. En lugar de realizar una asignación ciega que podría destruir el

trabajo del cruce, el operador selecciona un gen aleatorio y explora posibles clústeres de destino alternativos. La asignación se realiza siguiendo una estrategia condicional: el gen muta al primer clúster que reporte una infactibilidad estricta de 0. De este modo, la mutación inyecta diversidad en la población garantizando en todo momento que las soluciones se mantengan dentro del espacio de búsqueda factible.

```
function MUTATE(current_pop,  $p_{mut}$ )
   $n \leftarrow$  tamaño del cromosoma
   $k \leftarrow$  número de clusters
  for cada sol en current_pop do
    cambiado  $\leftarrow$  falso
    for  $i \leftarrow 0$  hasta  $n - 1$  do
      if random()  $< p_{mut}$  then
        orig_c  $\leftarrow$  sol[ $i$ ]                                ▷ Cluster original
        mejor_delta  $\leftarrow \infty$ 
        mejor_cluster  $\leftarrow -1$ 
        for  $c \leftarrow 0$  hasta  $k - 1$  do
          if  $c \neq orig\_c$  then
            delta  $\leftarrow$  CALCULATEINFEASIBILITYDELTA(sol,  $i$ , orig_c,  $c$ )
            if delta  $<$  mejor_delta then
              mejor_delta  $\leftarrow$  delta
              mejor_cluster  $\leftarrow$   $c$ 
            end if
          end if
        end for
        if mejor_cluster  $\neq -1$  y mejor_delta  $\leq 0$  then
          sol[ $i$ ]  $\leftarrow$  mejor_cluster
          cambiado  $\leftarrow$  verdadero
        end if
      end if
    end for
    if cambiado then
      REPAIRSOLUTION(sol)
    end if
  end for
end function
```

Tras generar la descendencia, se actualiza la población. En modo generacional (AGG) y meméticos, se sustituye toda la población por la descendencia, pero se asegura que el mejor padre sobreviva si supera al mejor hijo (elitismo). En modo AGE (reemplazo estacionario), solo se reemplazan los dos peores individuos de la población si los dos nuevos hijos son mejores que ellos. El pseudocódigo para este operador lo vemos en la siguiente lección.

3. Estructura de los algoritmos

A continuación, se detalla el esquema de funcionamiento principal de los algoritmos desarrollados. Todos ellos comparten la fase de inicialización, donde se genera una población (comprobando con el operador de reparación que no deja clusters vacíos), pero difieren significativamente en sus ciclos de evolución y reemplazo.

3.1. Algoritmos Genéticos Generacionales (AGG)

En el modelo generacional, la población intermedia (descendencia) tiene el mismo tamaño que la población original. En cada iteración, se seleccionan tantos padres como tamaño tenga la población mediante múltiples torneos. A esta población intermedia se le aplica el cruce (emparejando a los individuos de dos en dos de forma secuencial) y posteriormente la mutación poblacional.

Para el reemplazo, el AGG implementa un mecanismo de elitismo estricto para no perder la mejor solución encontrada. Una vez evaluada toda la descendencia, se compara el fitness del mejor hijo generado con el del mejor padre de la generación anterior. Si el mejor hijo resulta ser peor que el mejor padre, se localiza al peor individuo de la descendencia y se sustituye directamente por el padre elite. Tras este chequeo, la nueva población sustituye por completo a la anterior.

function RUN_AGG*population* $\leftarrow$ INITIALIZEPOPULATION*pop_fitness* $\leftarrow$ EVALUATEPOPULATION(*population*)**while** *current_evals* < *max_evals* **do***offspring* $\leftarrow$ SELECTION $\triangleright$ *pop_size* torneos de tamaño 3CROSSOVER(*offspring*) $\triangleright$ Cruce por pares, prob. $p_c = 0,8$ MUTATE(*offspring*) $\triangleright$ Por gen, prob. $p_m = 0,1/N$ *off_fitness* $\leftarrow$ EVALUATEPOPULATION(*offspring*)

REPLACEMENT_AGG

generations += 1**end while****return** *population*[arg mín(*pop_fitness*)]**end function**

function REPLACEMENT_AGG*best_parent* $\leftarrow$ GETBESTINDIVIDUAL(*pop_fitness*)*best_off* $\leftarrow$ GETBESTINDIVIDUAL(*off_fitness*)**if** *off_fitness*[*best_off*] > *pop_fitness*[*best_parent*] **then** $\triangleright$ Elitismo*worst_off* $\leftarrow$ GETWORSTINDIVIDUAL(*off_fitness*)*offspring*[*worst_off*] $\leftarrow$ *population*[*best_parent*]*off_fitness*[*worst_off*] $\leftarrow$ *pop_fitness*[*best_parent*]**end if***population* $\leftarrow$ *offspring***end function**

3.2. Algoritmos Genéticos Estacionarios (AGE)

El modelo estacionario plantea un enfoque mucho más conservador y de convergencia más rápida. En cada iteración, se seleccionan únicamente dos padres mediante torneo, generando de forma exclusiva dos descendientes que son sometidos a cruce y mutación.

A diferencia del AGG, no existe un reemplazo masivo. Los dos hijos generados compiten directamente con los dos peores individuos de la población actual. La inserción se realiza de forma secuencial: el primer hijo solo sustituirá al peor individuo de la población si su fitness es estrictamente inferior (mejor). A continuación, se vuelve a buscar al peor individuo actual (que podría haber cambiado tras la primera inserción) y el segundo hijo intenta sustituirlo bajo el mismo criterio de mejora estricta.

```
function RUN_AGE
  population  $\leftarrow$  INITIALIZEPOPULATION
  pop_fitness  $\leftarrow$  EVALUATEPOPULATION(population)
  while current_evals < max_evals do
    offspring  $\leftarrow$  SELECTION                                ▷ Solo 2 torneos
    CROSSOVER(offspring)
    MUTATE(offspring)
    off_fitness  $\leftarrow$  EVALUATEPOPULATION(offspring)
    REPLACEMENT_AGE
    generations += 1
  end while
  return population[arg mín(pop_fitness)]
end function
```

```
function REPLACEMENT_AGE
  worst1  $\leftarrow$  GETWORSTINDIVIDUAL(pop_fitness)
  if off_fitness[0] < pop_fitness[worst1] then
    population[worst1]  $\leftarrow$  offspring[0]
    pop_fitness[worst1]  $\leftarrow$  off_fitness[0]
  end if
  worst2  $\leftarrow$  GETWORSTINDIVIDUAL(pop_fitness)                ▷ Recalculado
  if off_fitness[1] < pop_fitness[worst2] then
    population[worst2]  $\leftarrow$  offspring[1]
    pop_fitness[worst2]  $\leftarrow$  off_fitness[1]
  end if
end function
```

```
function GETWORSTINDIVIDUAL(fitnesses)
  worst  $\leftarrow$  0
  for i  $\leftarrow$  1 to |fitnesses| - 1 do
    if fitnesses[i] > fitnesses[worst] then
      worst  $\leftarrow$  i
    end if
  end for
  return worst
end function
```

3.3. Algoritmos Meméticos (AM)

Los Algoritmos Meméticos implementados toman como base estructural el Algoritmo Genético Generacional (AGG), pero incorporan una fase de intensificación o Búsqueda Local Suave (BLS) para refinar las soluciones. Para mantener un equilibrio entre exploración y explotación, esta hibridación se activa periódicamente de forma determinista: exactamente cada 10 generaciones.

El alcance de esta mejora local viene definido por tres variantes implementadas:

- **AM-All:** Se aplica la búsqueda local exhaustivamente a todos y cada uno de los individuos de la población actual.
- **AM-Rand:** Se selecciona aleatoriamente a los individuos a optimizar. Se recorre la población y cada solución tiene un 10 % de probabilidad uniforme de ser sometida a la búsqueda local.
- **AM-Best:** Se aplica un enfoque puramente elitista. Se ordena la población según su fitness (de mejor a peor) y la búsqueda local se ejecuta únicamente sobre el 10 % de los mejores individuos.

Búsqueda Local Suave (BLS)

La optimización local está acotada para no agotar el presupuesto de evaluaciones del algoritmo principal. Para su control, se establecen dos límites fundamentales por cada individuo optimizado: un máximo de 100 evaluaciones de la función objetivo, y una condición de parada temprana que aborta la búsqueda si se acumulan $\epsilon = \lfloor 0,1 \cdot N \rfloor$ fallos, donde N es el tamaño de la población.

El procedimiento genera una permutación aleatoria de los índices para recorrerlos sin sesgo de orden. Por cada gen evaluado, se iteran todas las posibles asignaciones a clústeres distintos al actual. El criterio de aceptación sigue una estrategia *First-Improvement* guiada exclusivamente por la función objetivo: si el cambio supone una mejora estricta del fitness, se consolida y se pasa al siguiente gen. El proceso concluye cuando se alcanza el límite de evaluaciones, el límite de fallos, o se ha recorrido por completo la solución.

```
function APPLYMEMETIC
  if AM_ALL then
    for all  $i \in \{0, \dots, pop\_size - 1\}$  do
      if current_evals < max_evals then
        SOFTLOCALSEARCH(i)
      end if
    end for
  else if AM_RANDOM then
    for all  $i \in \{0, \dots, pop\_size - 1\}$  do
      if current_evals < max_evals and rand() ≤ 0,1 then
        SOFTLOCALSEARCH(i)
      end if
    end for
  else if AM_BEST then
    best_indices ← índices ordenados por pop_fitness ascendente
    for  $j \leftarrow 0$  to  $\lfloor 0,1 \cdot pop\_size \rfloor - 1$  do
      if current_evals < max_evals then
        SOFTLOCALSEARCH(best_indices[j])
      end if
    end for
  end if
end function
```

```
function SOFTLOCALSEARCH(pop_index)
  S ← population[pop_index], best_fit ← pop_fitness[pop_index]
  RSI ← RANDOMSHUFFLE(0, ..., N - 1)
   $\epsilon \leftarrow \lfloor 0,1 \cdot pop\_size \rfloor$ , fallos ← 0, i ← 0, evals ← 0, newsol ← S
  while fallos <  $\epsilon$  and i < N and evals < 100 do
     $p \leftarrow RSI[i]$ , oldvalue ← S[p]
    for all  $val \in \{0, \dots, k - 1\} \setminus \{oldvalue\}$  do
      if evals ≥ 100 then break
      end if
      newsol[p] ← val
      fit ← fitness(newsol); evals += 1
      if fit < best_fit then
        S ← newsol; best_fit ← fit
      else
        newsol[p] ← S[p]
      end if
    end for
    if S[p] = oldvalue then
      fallos += 1
    end if
    i += 1
  end while
  population[pop_index] ← S; pop_fitness[pop_index] ← best_fit
end function
```

4. Estructura del código

El proyecto sigue una arquitectura basada en la plantilla proporcionada en PRADO, con adaptaciones y extensiones específicas para el problema PAR y el análisis de resultados. El directorio principal `software/` se organiza de la siguiente manera:

- `common/`: Contiene las clases base y utilidades generales (`mh.h`, `problem.h`, `solution.h`, etc.). Estos archivos proporcionan la interfaz fundamental de la que heredan nuestros algoritmos y representaciones.
- `inc/` y `src/`: Directorios que separan, respectivamente, las cabeceras (`.h`) y las implementaciones (`.cpp`) de las clases desarrolladas para esta práctica. Incluyen la definición del problema (`problemPAR`) y la lógica de todos los algoritmos (`greedy`, `geneticalgorithm`, `localsearch`, `randomsearch`) implementando el método `optimize` para cada uno.
- `bin/`: Carpeta destino donde el compilador genera los archivos objeto (`.o`) y el ejecutable final (`par_solver`).
- `data/`: Directorio que almacena los conjuntos de datos y los archivos de restricciones a evaluar.
- **Archivos en la raíz:**
 - `main.cpp`: Punto de entrada del programa que gestiona los parámetros por consola y lanza la optimización.
 - `Makefile`: Archivo de configuración para automatizar la compilación.
 - `run_all_tests.sh`: Script en Bash para la ejecución en lote de toda la batería de pruebas.
 - `graf.py`: Script en Python para la generación automática de *boxplots* a partir de los resultados.
 - `wilcoxon.py`: Script en Python que realiza el test de Wilcoxon.
 - Archivos `.csv` y directorio `graficas/`: Salidas generadas tras la ejecución de los experimentos.

4.1. Manual de compilación

Para compilar el proyecto se ha incluido un `Makefile` configurado para el compilador `g++`. Se utiliza el estándar C++17 y el flag de máxima optimización `-O3` para acelerar las ejecuciones.

Para generar el ejecutable, basta con abrir un terminal en el directorio raíz del proyecto y ejecutar el comando:

```
$ make
```

Esto compilará los archivos fuente y generará el ejecutable `par_solver` dentro de la carpeta `bin/`. Si se desea limpiar el entorno eliminando los archivos compilados, se puede ejecutar:

```
$ make clean
```

4.2. Manual de ejecución

El software ofrece flexibilidad para ejecutarse de forma individual (para pruebas concretas) o de forma automatizada (para extraer todos los resultados de la memoria).

1. Ejecución individual

El ejecutable principal admite hasta 4 parámetros por línea de comandos, siguiendo la sintaxis:

```
$ ./bin/par_solver <dataset_base_path> <k_clusters> <algorithm> [seed]
```

- `<dataset_base_path>`: Ruta base del archivo de datos (ej. `data/glass_set`). El programa buscará automáticamente su archivo de restricciones asociado.
- `<k_clusters>`: Número de clusters a formar (típicamente 15 o 30).
- `<algorithm>`: Algoritmo a ejecutar (`random`, `greedy`, `local`, `agg-un`, `agg-sf`, `age-un`, `age-sf`, `am-all`, `am-rand`, `am-best`, o `all` para ejecutarlos todos secuencialmente).
- `[seed]` (*Opcional*): Semilla para el generador de números aleatorios. Si no se especifica, el programa realiza 5 ejecuciones independientes usando un conjunto de semillas por defecto.

Ejemplo de ejecución: Ejecutar todos los algoritmos sobre el dataset *zoo* con $k = 15$ y semilla 42:

```
$ ./bin/par_solver data/zoo_set 15 all 42
```

2. Ejecución automatizada (Batería de pruebas)

Para replicar los resultados mostrados en esta memoria, se ha desarrollado el script de bash `run_all_tests.sh`. Este script lanza internamente el programa para los datasets Bupa, Glass y Zoo (con $k = 15$ y $k = 30$) empleando las 10 semillas.

```
$ ./run_all_tests.sh
```

Al finalizar, el programa genera automáticamente los archivos `det_results.csv` (resultados detallados) y `avg_results.csv` (medias de tiempo, fitness y evaluaciones).

3. Generación de Gráficas

Una vez generados los archivos CSV, se pueden obtener los *boxplots* que comparen la estabilidad de los algoritmos ejecutando el script de Python:

```
$ python3 graf.py
```

Este comando leerá el archivo de resultados detallados y exportará las imágenes PNG dentro de la carpeta `graficas/`.

5. Experimentación y Análisis de Resultados

5.1. Descripción de los casos de prueba

Para evaluar el rendimiento y la robustez de los algoritmos implementados, se ha seleccionado un banco de pruebas compuesto por tres conjuntos de datos (*datasets*) de características variadas. Para cada uno de estos conjuntos, se ha abordado el problema de particionamiento considerando dos valores distintos para el número de clusters a formar: $k = 15$ y $k = 30$.

Las características principales de los conjuntos de datos, incluyendo el número total de instancias (n) y el número de atributos numéricos continuos que definen cada instancia (d), se resumen en la siguiente tabla:

Dataset (.dat)	Instancias (n)	Atributos (d)
bupa_set	345	5
glass_set	214	9
zoo_set	101	16

Cuadro 1: Características de los datasets empleados.

Junto a cada conjunto de datos y para cada valor de k , se proporciona un archivo de restricciones asociado (`_const_15.dat` y `_const_30.dat`), que define las condiciones estructurales *Must-Link* y *Cannot-Link* que los algoritmos deben intentar satisfacer.

5.2. Parámetros de ejecución

Dado el carácter estocástico de la gran mayoría de los algoritmos evaluados (Búsqueda Aleatoria, Búsqueda Local, Algoritmos Genéticos y Algoritmos Meméticos), es necesario definir un entorno de pruebas riguroso que permita extraer conclusiones con significancia estadística. La configuración del marco experimental es la siguiente:

- **Criterio de parada:** Los algoritmos estocásticos (Búsqueda Aleatoria, Búsqueda Local y todas las variantes Genéticas y Meméticas) se ejecutan utilizando como criterio de parada un límite máximo estricto de 100 000 evaluaciones de la función objetivo. Por su parte, el algoritmo *Greedy* es de naturaleza constructiva y finaliza cuando el procedimiento iterativo de asignación de instancias converge, por lo que no requiere un número máximo de evaluaciones.
- **Parámetros de los Algoritmos Poblacionales:** Para los Algoritmos Genéticos (AGG y AGE) y sus variantes Meméticas, se ha establecido un tamaño de población de 50 individuos. Las probabilidades de aplicación de los operadores genéticos se han fijado en los valores estándar para garantizar un equilibrio entre exploración y explotación (por ejemplo, una probabilidad de cruce del 70 % y una probabilidad de mutación del 10 % por cromosoma).
- **Parámetros de la hibridación Memética:** En las variantes híbridadas, la fase de intensificación (Búsqueda Local Suave) se aplica sistemáticamente cada

10 generaciones. En dicha fase, se dispone de un presupuesto interno de 100 evaluaciones a repartir. En la variante AM-All se aplica a los 50 individuos de la población; mientras que en AM-Best y AM-Rand se aplica de forma selectiva a un subconjunto de 5 individuos (el 10 % de la población).

- **Inicialización del generador aleatorio:** Para mitigar el sesgo introducido por la aleatoriedad en la inicialización de las poblaciones iniciales o en la aplicación de los operadores de variación, se han ejecutado 10 iteraciones independientes para cada combinación de algoritmo, *dataset* y valor de *k*.

Para garantizar la validez del análisis experimental, las ejecuciones se controlan mediante un conjunto de semillas consecutivas inicializado en el valor 1000 (es decir, $\{1000, 1001, \dots, 1009\}$), utilizadas para inicializar el generador de números aleatorios mediante la función `Random::seed()`. De este modo, la ejecución *i*-ésima de cada algoritmo utiliza la misma semilla, lo que permite realizar comparaciones emparejadas bajo idénticas condiciones iniciales. Esta estrategia asegura tanto la reproducibilidad de los experimentos como la validez de los contrastes estadísticos realizados posteriormente.

- **Registro de resultados:** Durante cada ejecución, el sistema ha registrado de manera automatizada el *fitness* alcanzado, la desviación intra-clúster, la *infeasibility* final, el número real de evaluaciones consumidas y el tiempo de cómputo en segundos.

5.3. Resultados obtenidos

En esta sección se presentan los resultados obtenidos en los experimentos realizados. Las tablas siguientes muestran los valores medios obtenidos a partir de dichas 10 ejecuciones. En concreto, se reportan las medias del valor de la función objetivo *fitness*, la desviación intra-cluster, la *infeasibility* (número de restricciones violadas), el número de evaluaciones de la función objetivo realizadas y el tiempo de ejecución en segundos.

Además de la presentación de los valores medios, se incluirán más tarde¹ diagramas *boxplots* que permiten visualizar la distribución de los resultados obtenidos en las 10 ejecuciones realizadas para cada algoritmo.

¹en la sección 5.4.5: Análisis de la variabilidad de los resultados

Algoritmo	Fitness	Distancia	Incumplimiento	Evaluaciones	Tiempo (s)
Random	0.34	0.25	490.7	100000	12.41
Greedy	0.24	0.24	17.2	1	0.02
Local	0.18	0.15	168.2	85442.6	10.26
AGG-UN	0.23	0.22	75.7	100000	16.15
AGG-SF	0.23	0.22	72.7	100000	16.22
AGE-UN	0.23	0.22	62.5	100000	16.13
AGE-SF	0.23	0.22	65.7	100000	16.29
AM-All	0.21	0.18	169.1	100000	15.89
AM-Rand	0.19	0.16	160.4	100000	15.98
AM-Best	0.19	0.16	161.2	100000	16.13

Cuadro 2: Resultados medios para el dataset *Bupa* con $k = 15$.

Algoritmo	Fitness	Distancia	Incumplimiento	Evaluaciones	Tiempo (s)
Random	0.28	0.23	567.3	100000	23.87
Greedy	0.23	0.23	3.5	1	0.03
Local	0.13	0.1	346.7	99781.7	23.09
AGG-UN	0.22	0.2	256.5	100000	29.46
AGG-SF	0.22	0.2	258	100000	29.69
AGE-UN	0.22	0.2	164.4	100000	29.47
AGE-SF	0.23	0.21	213.1	100000	29.73
AM-All	0.17	0.14	350.3	100000	29.13
AM-Rand	0.15	0.12	319.8	100000	29.31
AM-Best	0.14	0.11	327.8	100000	29.32

Cuadro 3: Resultados medios para el dataset *Bupa* con $k = 30$.

Algoritmo	Fitness	Distancia	Incumplimiento	Evaluaciones	Tiempo (s)
Random	0.52	0.4	173.5	100000	6.41
Greedy	0.34	0.34	0	1	0.01
Local	0.22	0.19	47.2	36397	2.07
AGG-UN	0.32	0.3	26.3	100000	5.85
AGG-SF	0.33	0.31	28	100000	5.93
AGE-UN	0.33	0.32	18.1	100000	5.87
AGE-SF	0.33	0.32	17.1	100000	5.99
AM-All	0.24	0.2	49.1	100000	5.67
AM-Rand	0.23	0.2	47.2	100000	5.8
AM-Best	0.22	0.19	42.9	100000	5.75

Cuadro 4: Resultados medios para el dataset *Glass* con $k = 15$.

Algoritmo	Fitness	Distancia	Incumplimiento	Evaluaciones	Tiempo (s)
Random	0.42	0.35	218.7	100000	11.29
Greedy	73333.5	73333.5	0	1	0.02
Local	0.16	0.13	103.6	76518.1	8.14
AGG-UN	0.28	0.26	68.4	100000	10.75
AGG-SF	0.27	0.25	65.4	100000	10.8
AGE-UN	0.31	0.28	71.8	100000	10.73
AGE-SF	0.29	0.27	71.6	100000	10.80
AM-All	0.18	0.15	105.2	100000	10.55
AM-Rand	0.16	0.13	94.5	100000	10.73
AM-Best	0.15	0.12	91.1	100000	10.76

Cuadro 5: Resultados medios para el dataset *Glass* con $k = 30$.

Algoritmo	Fitness	Distancia	Incumplimiento	Evaluaciones	Tiempo (s)
Random	1.47	1.27	37	100000	1.31
Greedy	13334.20	13334.2	0	1	$1,96 \times 10^{-3}$
Local	0.53	0.5	6.3	13951.9	0.12
AGG-UN	1.01	0.95	11.2	100000	0.90
AGG-SF	0.98	0.93	8.6	100000	0.97
AGE-UN	0.91	0.88	5.8	100000	0.92
AGE-SF	0.87	0.84	5.4	100000	0.95
AM-All	0.53	0.5	5.1	100000	0.88
AM-Rand	0.52	0.48	5.9	100000	0.9
AM-Best	0.51	0.47	7	100000	0.88

Cuadro 6: Resultados medios para el dataset *Zoo* con $k = 15$.

Algoritmo	Fitness	Distancia	Incumplimiento	Evaluaciones	Tiempo (s)
Random	1.03	0.9	48.7	100000	2.03
Greedy	140000	140000	0	1	$3,41 \times 10^{-3}$
Local	0.29	0.24	17.5	23325.4	0.25
AGG-UN	0.65	0.59	22.4	100000	1.05
AGG-SF	0.58	0.52	24.4	100000	1.1
AGE-UN	0.51	0.48	11.8	100000	1.04
AGE-SF	0.61	0.56	17.3	100000	1.08
AM-All	0.3	0.25	18.4	100000	1.02
AM-Rand	0.29	0.25	15.5	100000	1.02
AM-Best	0.29	0.25	15.1	100000	1.04

Cuadro 7: Resultados medios para el dataset *Zoo* con $k = 30$.

5.4. Análisis de resultados

Una vez presentados los resultados promedios de las ejecuciones, procedemos a realizar un análisis exhaustivo del comportamiento de los diferentes algoritmos im-

plementados. En esta sección se evaluará cómo la introducción de enfoques poblacionales (Algoritmos Genéticos) y su posterior hibridación (Algoritmos Meméticos) impactan en la calidad de las soluciones, interpretando los resultados en base a los mecanismos internos de exploración y explotación de cada técnica.

5.4.1. Comparación general entre algoritmos

A partir de los resultados observados, se evidencia una clara jerarquía en el rendimiento de los distintos algoritmos. La *Búsqueda Aleatoria* sigue estableciéndose como el límite inferior de calidad, obteniendo consistentemente los peores valores de *fitness* en todos los escenarios (por ejemplo, 0,34 en *Bupa* con $k = 15$, Cuadro 2).

La introducción de los Algoritmos Genéticos (AGG y AGE) supone una mejora drástica y robusta frente a la aleatoriedad y a las fallas catastróficas del enfoque *Greedy*. Los genéticos puros logran estabilizar el *fitness* en valores muy competitivos (en torno a 0,22 – 0,23 en *Bupa* $k = 30$, y 0,33 en *Glass* $k = 15$). Sin embargo, es en los Algoritmos Meméticos donde encontramos los mejores resultados globales, logrando igualar y en la mayoría de casos superar a la *Búsqueda Local* tradicional.

Dentro de las variantes meméticas, se observa sistemáticamente en todas las tablas que AM-Best y AM-Rand superan a AM-All. Por ejemplo, en *Bupa* con $k = 30$ (Cuadro 3), AM-Best alcanza un *fitness* de 0,14, frente al 0,17 de AM-All. Esto demuestra que refinar a todos los individuos de la población desperdicia evaluaciones, mientras que centrar la optimización local solo en los mejores individuos (AM-Best) o en un subconjunto exploratorio (AM-Rand) permite al algoritmo genético base iterar durante más generaciones y alcanzar mejores óptimos.

5.4.2. Impacto de las restricciones en la función objetivo

El análisis detallado de las componentes de la función objetivo (Distancia frente a Incumplimiento) revela cómo cada algoritmo gestiona el compromiso entre calidad geométrica y factibilidad relacional.

Al igual que ocurría en la Práctica anterior, para el algoritmo *Greedy* las penalizaciones por los clusters vacíos (como se refleja en los 73333,5 en *Glass* $k = 30$ o los 140000 en *Zoo* $k = 30$) siguen siendo irreparables. En contraposición, los Algoritmos Genéticos Puros (AGG y AGE) destacan por mantener niveles de incumplimiento (*infeasibility*) sorprendentemente bajos. En *Zoo* con $k = 15$ (Cuadro 6), las variantes AGG y AGE logran valores de incumplimiento moderados (entre 5,4 y 11,2), notablemente inferiores a los de *Random* (37) y *Local* (6,3). Este comportamiento se debe al operador de mutación basado en delta de infeasibility, que restringe los movimientos que empeoran la factibilidad, y al mecanismo de reparación posterior que evita clusters vacíos.

Resulta especialmente interesante el comportamiento de los Algoritmos Meméticos y la *Búsqueda Local*. Para obtener los valores más bajos de *fitness* total, estos métodos asumen un mayor grado de incumplimiento. En *Bupa* con $k = 30$ (Cuadro 3), mientras que AGG-UN tiene un incumplimiento de 256,5, AM-Best lo eleva hasta 327,8, pero logra reducir la distancia de 0,2 a 0,11. Esto indica que la búsqueda local inyectada en los meméticos mejora vorazmente la compacidad de los clústeres,

sacrificando estratégicamente algunas restricciones relacionales al comprobar que la ganancia geométrica compensa la penalización.

5.4.3. Esfuerzo computacional y convergencia

La distribución de las evaluaciones constituye un factor diferenciador clave. Mientras que la *Búsqueda Local* tradicional muestra convergencias tempranas formidables (deteniéndose en apenas 13951,9 evaluaciones medias en *Zoo* $k = 15$, Cuadro 6), todos los enfoques poblacionales (Genéticos y Meméticos) consumen el límite estricto de 100000 evaluaciones por diseño.

La diferencia radica en cómo invierten este presupuesto computacional. En los genéticos puros, las 100000 evaluaciones se traducen en un alto número de generaciones evolutivas. En los meméticos, especialmente en AM-All, el límite de evaluaciones se agota muy rápidamente en refinar individuos, lo que restringe severamente el número de saltos generacionales. Esta es la razón técnica que justifica los datos de las tablas: al limitar la hibridación al 10 % de la población (AM-Best), se equilibra el esfuerzo, permitiendo que el bucle evolutivo principal siga funcionando mientras se explota a los individuos más prometedores.

5.4.4. Comparación entre conjuntos de datos y dimensionalidad

Al contrastar los resultados entre $k = 15$ y $k = 30$, se consolidan los efectos de la dimensionalidad anticipados en experimentaciones previas. Aumentar el número de clústeres facilita la reducción del término de distancia intra-clúster en todos los métodos, pero incrementa exponencialmente la dificultad para mantener la coherencia de las restricciones (*Must-Link* y *Cannot-Link*).

Este fenómeno se aprecia de forma nítida en el dataset *Zoo*. Para AM-Best, al pasar de $k = 15$ a $k = 30$ (Cuadros 6 y 7), la distancia media se reduce de 0,47 a 0,25, pero las penalizaciones por incumplimiento crecen de 7 a 15,1.

Por otro lado, comparando los operadores de cruce (Uniforme frente a Segmento) en diferentes conjuntos de datos, los resultados tabulados muestran una competencia muy ajustada. En conjuntos con baja tasa de error como *Zoo* $k = 30$ (Cuadro 7), el cruce uniforme (AGG-UN) logra reducir el incumplimiento frente al de segmento (AGG-SF, 22,4 frente a 24,4), aunque con un *fitness* ligeramente peor (0,65 frente a 0,58). Esto sugiere que en este dataset el cruce por segmento favorece la exploración geométrica a costa de violar más restricciones, mientras que el uniforme mantiene mejor la factibilidad aunque con menor compacidad.

5.4.5. Análisis de la variabilidad de los resultados

Como se ha mencionado previamente, con el objetivo de evaluar la estabilidad de los algoritmos, cada configuración se ejecutó 10 veces sobre el dataset *bupa_set* para distintos valores de k . A partir de estas ejecuciones se construyeron diagramas de caja (*boxplots*), que permiten analizar la distribución del *fitness*, su dispersión y la presencia de posibles valores atípicos (*outliers*).

Dataset `bupa_set` En la Figura 1 se muestra la distribución del *fitness* para las distintas variantes de algoritmos evaluadas, considerando $k = 15$ y $k = 30$.

En ambos valores de k se observa, en general, una baja dispersión en los resultados, lo que indica un comportamiento estable entre ejecuciones para la mayoría de los algoritmos. No obstante, existen diferencias claras tanto en calidad de las soluciones como en variabilidad.

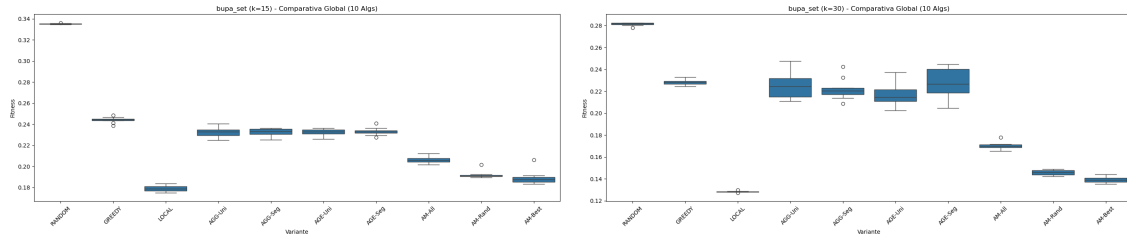


Figura 1: Distribución del fitness para el dataset *Bupa* con $k = 15$ (izq) y $k = 30$ (dcha)

Para $k = 15$, *Random* presenta claramente los peores resultados, seguido de *Greedy*, mientras que *Local* alcanza los mejores valores de *fitness* con muy baja variabilidad. Los algoritmos genéticos (AGG y AGE) quedan en una posición intermedia muy compacta, todos alrededor de $\sim 0,233$ con baja dispersión, y los meméticos mejoran estos resultados, destacando AM-Best y AM-Rand, como puede verse claramente en la comparativa global.

Este comportamiento se aprecia con mayor detalle en la Figura 2. Para $k = 15$, las cuatro variantes genéticas son prácticamente indistinguibles, con medianas entre 0,232 y 0,234 y cajas muy estrechas, lo que indica una convergencia homogénea independientemente del operador de cruce o esquema evolutivo. Para $k = 30$, emergen diferencias más claras: AGE-UN obtiene la mejor mediana ($\sim 0,215$) y AGE-Seg presenta la mayor variabilidad y peores resultados, lo que sugiere que el cruce por segmento fijo no se adapta bien al mayor espacio de búsqueda en el esquema estacionario.

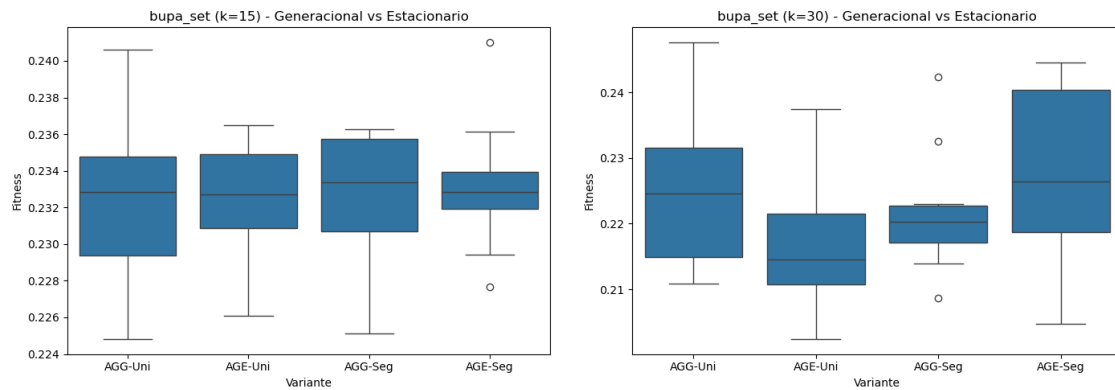


Figura 2: Comparativa Generacional vs Estacionario para el dataset *Bupa* con $k = 15$ (izq) y $k = 30$ (dcha)

Finalmente, en la Figura 3, como puede observarse, se distinguen claramente las diferencias entre estrategias meméticas y la búsqueda Local.

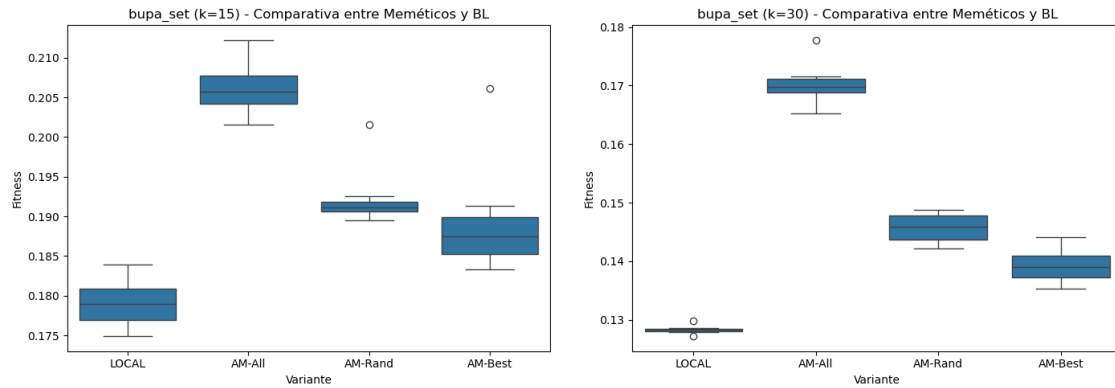


Figura 3: Comparativa entre Meméticos y Búsqueda Local para el dataset *Bupa* con $k = 15$ (izq) y $k = 30$ (dcha)

Para $k = 15$, *Local* sigue siendo el algoritmo más competitivo ($\sim 0,179$), lo que indica que con este dataset y valor de k la exploración poblacional no compensa frente a una búsqueda local pura con el mismo presupuesto. AM-Best ($\sim 0,188$) y AM-Rand ($\sim 0,191$) se aproximan a *Local*, mientras que AM-All queda más rezagado ($\sim 0,206$).

Para $k = 30$, la brecha con *Local* ($\sim 0,128$) se amplía: AM-Best ($\sim 0,139$) y AM-Rand ($\sim 0,146$) son los más cercanos, mientras que AM-All ($\sim 0,170$) queda muy por detrás.

En conjunto, como se observa en todas las figuras, los algoritmos más intensivos en explotación (*Local* y meméticos) no solo logran mejores resultados, sino que también presentan menor variabilidad, evidenciando una convergencia más robusta.

Dataset *glass_set* En la Figura 4 se muestra la distribución del *fitness* para las distintas variantes de algoritmos evaluadas, considerando $k = 15$ y $k = 30$.

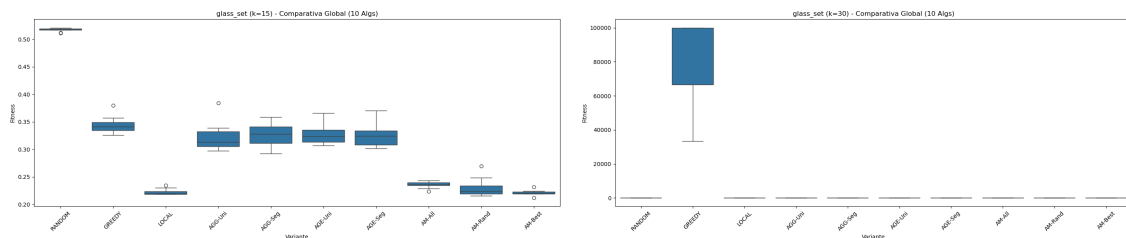


Figura 4: Distribución del fitness para el dataset *Glass* con $k = 15$ (izq) y $k = 30$ (dcha)

Para $k = 15$, el comportamiento general es muy parecido al anterior: *Random* presenta los peores resultados, seguido de *Greedy* y *Local* alcanza valores competitivos con baja dispersión. Los algoritmos genéticos (AGG y AGE) se sitúan en una franja intermedia entre 0,31 y 0,33, con AGG-UN mostrando la menor dispersión, mientras que los meméticos mejoran notablemente estos resultados, siendo AM-Best y AM-Rand los más destacados, con valores próximos a *Local*.

Para $k = 30$, la gráfica global presenta un comportamiento anómalo en *Greedy*, cuyo *fitness* alcanza valores muy altos, muy por encima del resto, debido a las penalizaciones por clusters vacíos. El resto de algoritmos, al explorar el espacio de

búsqueda más ampliamente, evitan estas soluciones y se concentran en una franja de fitness más cercana a 0, lo que distorsiona la escala visual de la comparativa global.

En la Figura 5 se aprecia la comparativa entre genéticos y meméticos: los meméticos se separan claramente de los genéticos puros, con AM-Best ($\sim 0,153$) y AM-Rand ($\sim 0,160$) muy por debajo de cualquier variante genética (0,265–0,320).

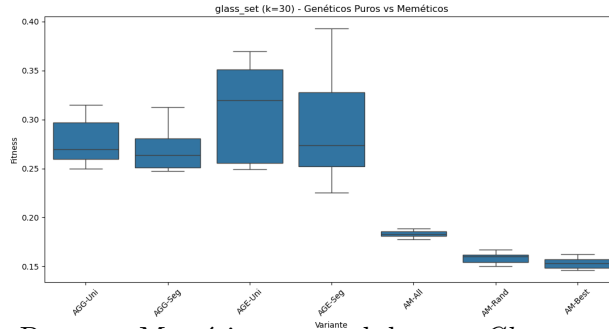


Figura 5: Genéticos Puros vs Meméticos para el dataset *Glass* con $k = 15$ (izq) y $k = 30$ (dcha)

Para $k = 15$, AGG-UN destaca con la mediana más baja ($\sim 0,312$) y menor dispersión, mientras que AGG-Seg presenta la mayor variabilidad. Para $k = 30$, AGG-Seg obtiene ahora la mejor mediana ($\sim 0,263$) con poca dispersión, mientras que AGE-UN muestra la mayor variabilidad y peores resultados ($\sim 0,320$), evidenciando que el esquema estacionario con cruce uniforme es especialmente inestable al aumentar k . Esto lo comprobamos mejor en la Figura (6).

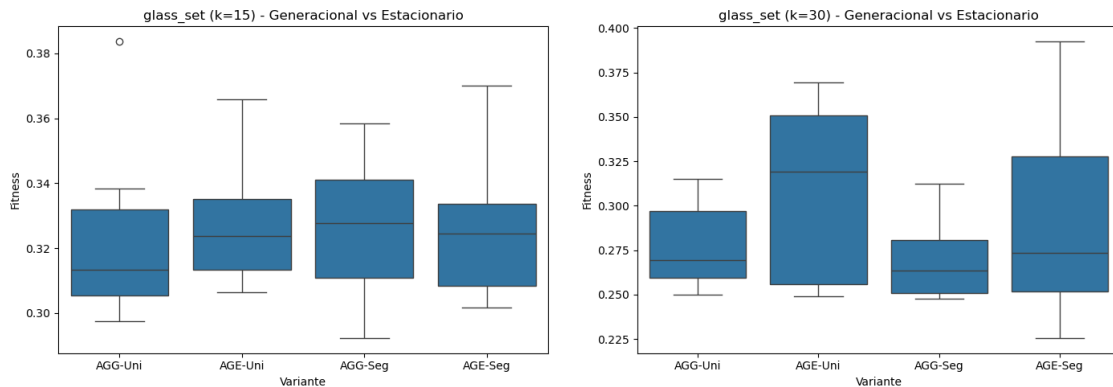


Figura 6: Comparativa Generacional vs Estacionario para el dataset *Glass* con $k = 15$ (izq) y $k = 30$ (dcha)

Finalmente, la Figura 7 detalla la comparativa entre estrategias meméticas y BL.

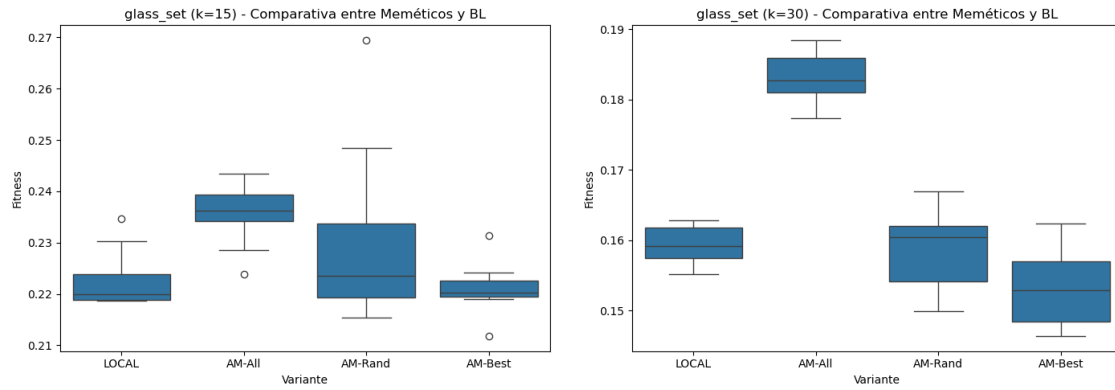


Figura 7: Comparativa entre Meméticos y BL para el dataset *Glass* con $k = 15$ (izq) y $k = 30$ (dcha)

En ambos valores de k se repite el mismo patrón entre meméticos: AM-Best obtiene los mejores resultados, seguido de AM-Rand, mientras que AM-All presenta el peor rendimiento. La razón es estructural: AM-All distribuye las 100 evaluaciones de BLS entre los 50 individuos, de modo que cada uno recibe apenas 2 evaluaciones de media, una intensificación insuficiente. AM-Best concentra esas mismas evaluaciones en los 5 mejores individuos, que parten ya de regiones prometedoras del espacio de búsqueda.

Para $k = 15$, AM-Best ($\sim 0,221$) y *Local* ($\sim 0,221$) son prácticamente equivalentes en mediana, aunque AM-Best presenta menor dispersión. AM-Rand muestra una varianza muy elevada, con algunos outliers próximos a 0,27, lo que indica inestabilidad en su selección aleatoria. AM-All ($\sim 0,236$) queda claramente por detrás.

Para $k = 30$, la ventaja de los meméticos selectivos se consolida: AM-Best ($\sim 0,153$) supera a *Local* ($\sim 0,160$) y AM-Rand ($\sim 0,159$) se sitúa a un nivel similar, mientras que AM-All ($\sim 0,183$) sigue siendo el peor entre los meméticos. La diversidad genética resulta decisiva para $k = 30$, donde el espacio de búsqueda es suficientemente grande como para que la BL pura quede atrapada en óptimos locales de menor calidad.

Dataset *zoo_set* En la Figura 8 se muestra la distribución del *fitness* para las distintas variantes de algoritmos evaluadas, considerando $k = 15$ y $k = 30$.

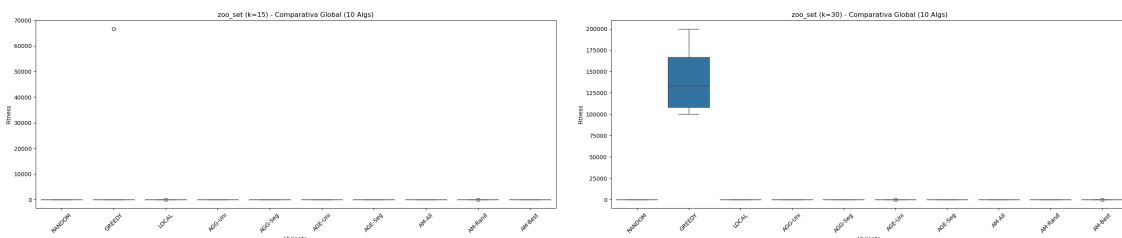


Figura 8: Distribución del fitness para el dataset *Zoo* con $k = 15$ (izq) y $k = 30$ (dcha)

Volvemos a observar lo que ocurría antes en *glass_set*, y es que en ambos valores de k , el comportamiento de *Greedy* con sus altas penalizaciones por clusters vacíos lleva a presentar valores de *fitness* muy altos, muy por encima del resto de algoritmos.

Por ello, se incluyen figuras adicionales con zoom que permiten comparar en detalle las variantes genéticas entre sí y frente a los meméticos.

La Figura 9 muestra la comparativa entre genéticos puros y meméticos, donde la ventaja de incorporar búsqueda local resulta especialmente evidente.

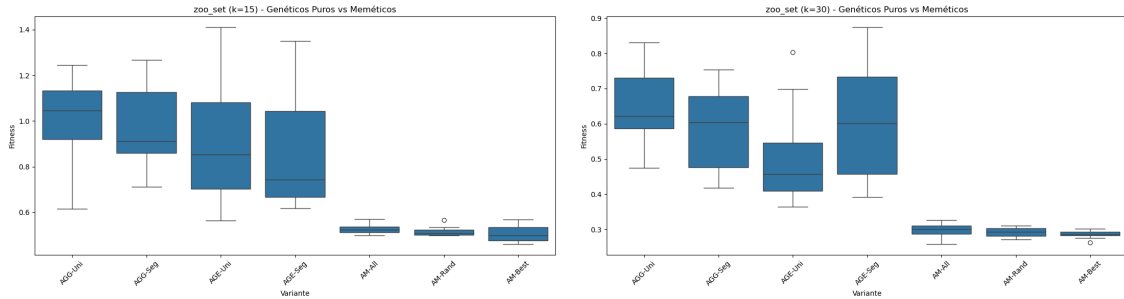


Figura 9: Genéticos Puros vs Meméticos para el dataset *Zoo* con $k = 15$ (izq) y $k = 30$ (dcha)

La mejora introducida por la hibridación con búsqueda local es notable en ambos valores de k . Para $k = 15$, los genéticos puros se concentran en una franja elevada de 0,62–1,15, mientras que los meméticos descienden fuertemente hasta 0,49–0,54, una reducción de más del 10% en *fitness*. Para $k = 30$, la separación es aún más pronunciada: los genéticos puros oscilan entre 0,45 y 0,75, mientras que los meméticos alcanzan valores muy competitivos de 0,27–0,30. En ambos casos, AM-Best es el que mayor beneficio obtiene de la intensificación local.

En la Figura 10 se comparan exclusivamente las variantes genéticas.

Para $k = 15$, las cuatro variantes genéticas presentan diferencias notables: los enfoques estacionarios (AGE-UN y AGE-SF) logran mejores resultados que los generacionales, situando sus medianas en torno a 0,85–0,90. Para $k = 30$, AGE-UN destaca claramente como la mejor opción pura, presentando la mediana más baja ($\sim 0,50$) y la menor dispersión. En contraste, las variantes generacionales (AGG) sufren más para optimizar este escenario, sugiriendo que la sustitución paulatina de la población del modelo estacionario preserva mejor las estructuras válidas en este conjunto de datos.

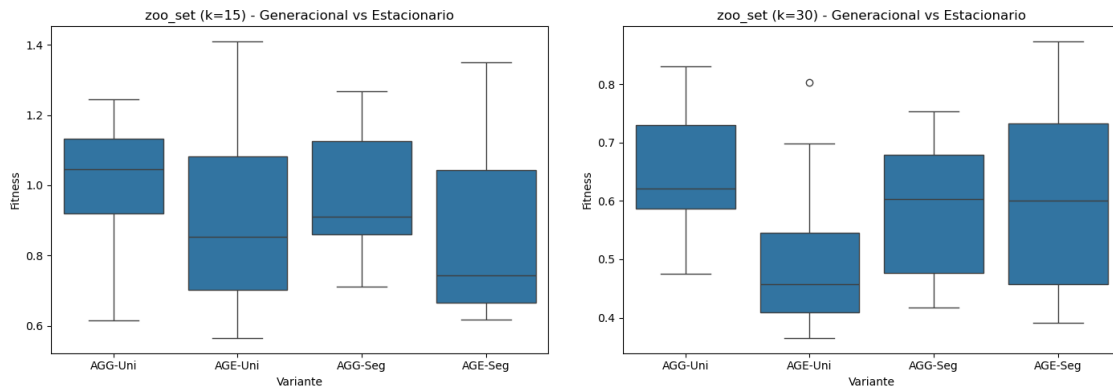


Figura 10: Comparativa Generacional vs Estacionario para el dataset *Zoo* con $k = 15$ (izq) y $k = 30$ (dcha)

Finalmente, la Figura 11 detalla la comparativa entre estrategias meméticas.

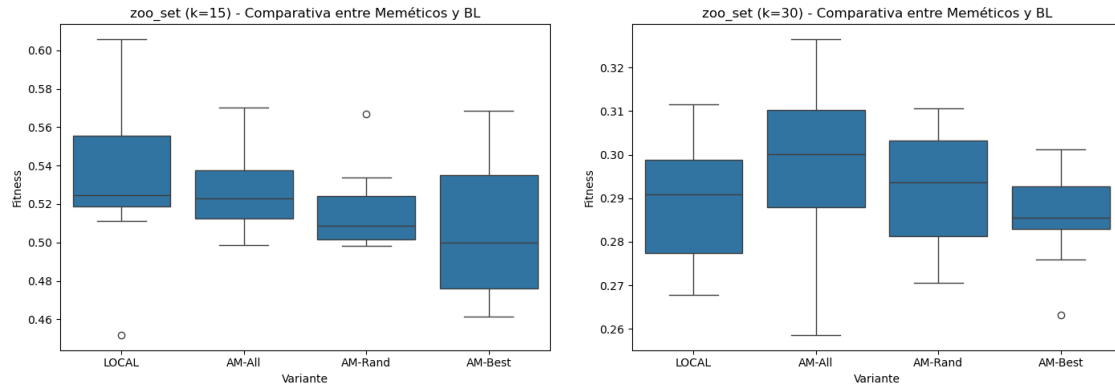


Figura 11: Comparativa entre Meméticos y BL para el dataset *Zoo* con $k = 15$ (izq) y $k = 30$ (dcha)

Para $k = 15$, AM-Best logra la mejor mediana global (en torno a 0,49), superando claramente a la Búsqueda Local (*Local*, $\sim 0,53$) en la calidad central de las soluciones. Sin embargo, al observar la dispersión, destaca que AM-Best presenta una varianza notablemente mayor. Esto indica que la hibridación selectiva permite al algoritmo alcanzar las regiones más óptimas del espacio de búsqueda (como demuestran sus mejores valores), pero el resultado final fluctúa más dependiendo de las condiciones iniciales (semilla). Por el contrario, *Local* presenta una menor varianza, lo que refleja un comportamiento muy estable pero indeseado: se estanca de forma consistente en óptimos locales de peor calidad. Las variantes AM-Rand y AM-All se sitúan en un punto intermedio, compactando la varianza pero con medianas ligeramente menos competitivas que AM-Best.

Para $k = 30$, pasa algo bastante interesante: la Búsqueda Local pura (*Local*) consigue la mejor mediana, superando un poco a los algoritmos meméticos. Esto significa que, en promedio, suele encontrar soluciones ligeramente mejores. Esto ocurre porque, en un espacio tan limitado como *Zoo*, la Búsqueda Local puede centrarse totalmente en mejorar una solución y exprimir al máximo cada zona (óptimos locales).

Por su parte, el algoritmo AM-Best destaca porque sus resultados son mucho más estables (tiene menos variabilidad). Aunque su mediana es un poco peor, casi siempre obtiene soluciones parecidas y bastante buenas. Esto pasa porque, en los meméticos, el cruce introduce cambios que a veces rompen estructuras que ya estaban bien optimizadas, lo que perjudica un poco el resultado final.

5.4.6. Eficiencia computacional y tiempos de ejecución

Un aspecto adicional que merece ser analizado es la eficiencia temporal de los algoritmos poblacionales. Observando la columna de tiempos de ejecución en las tablas, surge un patrón interesante: a pesar de que los algoritmos meméticos incorporan una fase de intensificación (Búsqueda Local Suave) que a priori podría intuirse como un cuello de botella, sus tiempos totales de cómputo resultan ser prácticamente idénticos a los de los algoritmos genéticos puros.

Por ejemplo, en el conjunto de datos *Bupa* con $k = 30$ (Cuadro 3), el algoritmo genético puro AGG-UN requiere una media de 29,46 segundos, mientras que la variante memética AM-Best finaliza en 29,32 segundos. Esta paridad temporal sugiere que el coste de consumir evaluaciones mediante saltos generacionales (lo que implica un gran sobrecoste en creación de descendencia, cruce, mutación y, sobre todo, ejecución constante del mecanismo de reparación de infactibilidades para toda la población) es computacionalmente equiparable a invertir esas mismas evaluaciones evaluando vecinos en la Búsqueda Local Suave (la cual modifica un único gen sin necesidad de recrear estructuras completas).

Por lo tanto, la verdadera fortaleza de las variantes meméticas (especialmente AM-Best y AM-Rand) en esta experimentación no reside en la reducción del tiempo bruto de ejecución, sino en la *rentabilidad* de la optimización. Al obtener resultados de *fitness* significativamente mejores e incuestionablemente superiores a nivel estadístico (por ejemplo, reduciendo el *fitness* medio de 0,22 a 0,14 en el citado caso de *Bupa* $k = 30$) consumiendo exactamente el mismo presupuesto temporal de CPU, la hibridación demuestra ser una arquitectura computacionalmente mucho más eficiente.

5.4.7. Conclusiones generales del análisis

A modo de síntesis, la experimentación y el análisis visual confirman que la hibridación algorítmica representa la estrategia más robusta y eficaz para abordar este problema de *clustering* con restricciones. Mientras que los algoritmos de trayectoria simple muestran problemas de convergencia prematura o incapacidad para reparar soluciones infactibles (como evidencia el estancamiento de *Greedy*), y los genéticos puros adolecen de falta de intensificación local, los Algoritmos Meméticos logran el equilibrio óptimo entre exploración y explotación.

En particular, la variante AM-Best se erige de forma unánime como la arquitectura superior del estudio. La aplicación selectiva de la Búsqueda Local Suave al 10 % de la población no solo reporta sistemáticamente los mejores valores de *fitness* y sorteja con mayor éxito las topologías más complejas ($k = 30$), sino que además lo hace invirtiendo exactamente el mismo presupuesto temporal, demostrando que optimizar a los mejores individuos es una táctica altamente rentable, computacionalmente eficiente y estadísticamente significativa.

Nota: Para dotar de rigor formal a estas conclusiones empíricas y confirmar que la superioridad de la arquitectura memética no es fruto del azar, a continuación se detalla el análisis estadístico realizado mediante el test no paramétrico de Wilcoxon.

6. Contraste de Hipótesis mediante el Test de Wilcoxon

A lo largo del análisis previo, tanto la inspección visual de las gráficas como los valores promedio han sugerido una clara superioridad de los Algoritmos Meméticos frente a los Algoritmos Genéticos puros. No obstante, con el fin de dotar a esta afirmación de mayor rigor estadístico y justificar la inclusión de la Búsqueda Local en el esquema evolutivo, se ha realizado un contraste de hipótesis emparejado entre el algoritmo generacional puro (AGG-UN) y la mejor variante memética (AM-Best).

Dado que ambos algoritmos han sido evaluados bajo condiciones estrictamente emparejadas (utilizando las mismas 10 semillas aleatorias en cada experimento) y que no puede asumirse *a priori* la normalidad de las distribuciones del *fitness*, no resulta apropiado aplicar un test paramétrico como el test de Student. En su lugar, se ha empleado el **test de los rangos signados de Wilcoxon**, un test no paramétrico idóneo para estas condiciones.

Las hipótesis consideradas en el estudio son:

- **Hipótesis nula (H_0):** No existen diferencias significativas entre las medianas del *fitness* obtenido por el algoritmo genético AGG-UN y el algoritmo memético AM-Best.
- **Hipótesis alternativa (H_1):** El algoritmo memético AM-Best obtiene valores de *fitness* significativamente menores (mejores) que el algoritmo genético AGG-UN.

Para la realización del contraste se ha desarrollado un *script* en Python utilizando la librería `scipy.stats`. El test se ha aplicado de forma independiente para cada conjunto de datos (*Bupa*, *Glass*, *Zoo*) y para cada valor de k (15 y 30), fijando un nivel de significación $\alpha = 0,05$. Los resultados obtenidos son los siguientes:

RESULTADOS DEL TEST DE WILCOXON (AGG-Uniform vs AM-Best)

Dataset: data/bupa_set | k: 15 | p-valor: 9.766e-04
Diferencia significativa (AM-Best es estadísticamente mejor)

Dataset: data/bupa_set | k: 30 | p-valor: 9.766e-04
Diferencia significativa (AM-Best es estadísticamente mejor)

Dataset: data/glass_set | k: 15 | p-valor: 9.766e-04
Diferencia significativa (AM-Best es estadísticamente mejor)

Dataset: data/glass_set | k: 30 | p-valor: 9.766e-04
Diferencia significativa (AM-Best es estadísticamente mejor)

Dataset: data/zoo_set | k: 15 | p-valor: 9.766e-04
Diferencia significativa (AM-Best es estadísticamente mejor)

Dataset: data/zoo_set | k: 30 | p-valor: 9.766e-04
Diferencia significativa (AM-Best es estadísticamente mejor)

Interpretación y conclusiones del contraste

En todos los escenarios analizados, los *p-valores* obtenidos mediante el test de Wilcoxon son claramente inferiores al nivel de significación establecido ($\alpha = 0,05$). Por tanto, los datos aportan evidencia estadística suficiente para rechazar la hipótesis nula (H_0) en favor de la hipótesis alternativa (H_1).

Un aspecto analítico interesante es la magnitud exacta de estos *p-valores*. En los seis escenarios, el test arroja un valor de $9,766 \times 10^{-4}$. Este es el valor probabilístico mínimo posible para una muestra emparejada de tamaño $N = 10$ ($1/2^{10} \approx 9,77 \times 10^{-4}$), lo que indica una dominancia absoluta: la variante AM-Best superó a AGG-UN en el 100 % de las semillas ejecutadas.

En definitiva, este contraste de hipótesis arroja un resultado concluyente: demuestra estadísticamente que la hibridación algorítmica es altamente efectiva en este problema. La incorporación de una Búsqueda Local Suave aplicada al 10 % de los mejores individuos de la población mejora sistemáticamente el proceso de optimización frente al uso exclusivo de los operadores de cruce y mutación tradicionales, justificando de manera rigurosa el desarrollo y adopción de arquitecturas meméticas.

7. Referencias y recursos utilizados

Para la realización de esta práctica y la elaboración de la presente memoria, se han consultado y utilizado los siguientes recursos:

- **Material de la asignatura:** Diapositivas, guión de prácticas y recursos de PRADO.
- **Herramientas de Inteligencia Artificial:** Se ha hecho uso de herramientas de IA generativa como apoyo en tareas de programación y redacción. En concreto, se han empleado para:
 - Resolver dudas puntuales de sintaxis en C++ y mejorar la organización del archivo `Makefile`.
 - Desarrollar el script de Python (`graf.py`), utilizando las librerías `pandas`, `matplotlib` y `seaborn`, para la generación automatizada de gráficos (*boxplots*) y el procesamiento de los datos almacenados en archivos `.csv`.
 - Asistir en el formateo, revisión y estructuración del documento final en $\text{\LaTeX}$.
- Documentación oficial de C++ (cppreference.com) para el manejo de estructuras de datos estándar, y documentación oficial de las librerías de Python empleadas para la visualización de datos.
- Para la realización del análisis estadístico de los resultados, en la aplicación del test de hipótesis de Wilcoxon, se ha utilizado el material de la asignatura *Inferencia Estadística*, dentro de la parte de Matemáticas del doble grado.

8. Anexo - Falta de espacio

Mecanismo de Reparación (`repairSolution`): Todo individuo generado tras el cruce se somete a una verificación estricta. Si algún clúster queda vacío (0 instancias asignadas), el método busca instancias pertenecientes a clústeres sobrepoblados (con más de una instancia) y prueba a reasignarlas al clúster vacío. El proceso sigue estos pasos:

1. Calcula la frecuencia de cada clúster en la solución.
2. Para cada clúster c sin elementos:
 - Recopila índices de instancias que provienen de clústeres con al menos dos elementos.
 - Mezcla el orden de esas instancias para evitar sesgos.
 - Prueba secuencialmente: mueve la instancia al clúster vacío y evalúa la infactibilidad global mediante el método `calculateInfeasibilityDelta()`.
 - Si el cambio $\Delta \leq 0$ (es decir, la infactibilidad no empeora), consolida el movimiento y pasa al siguiente clúster vacío. Esto permite mantener o mejorar la factibilidad sin necesidad de exigir infactibilidad global cero.
 - En caso contrario, deshace el cambio y prueba con la siguiente instancia.
3. Si no se encuentra ninguna reasignación con $\Delta \leq 0$, aplica un *fallback* forzado con la primera instancia candidata (garantiza que no queden clústeres vacíos, aunque pueda generar infactibilidad temporal; esta será tratada por la función de penalización en el fitness).

Este mecanismo asegura que la mayoría de las soluciones sean factibles en cuanto a cardinalidad de los clústeres, respetando al mismo tiempo las restricciones estructurales del problema.

A continuación, mostramos el pseudocódigo:

```
function REPAIRSOLUTION(sol)
   $k \leftarrow$  número de clusters
   $n \leftarrow$  longitud del cromosoma
   $contador \leftarrow$  vector de ceros de tamaño  $k$ 
  for  $i \leftarrow 0$  hasta  $n - 1$  do
     $contador[sol[i]] \leftarrow contador[sol[i]] + 1$ 
  end for
  for cada cluster  $c$  desde 0 hasta  $k - 1$  do
    if  $contador[c] = 0$  then
       $robable \leftarrow$  lista vacía
      for  $i \leftarrow 0$  hasta  $n - 1$  do
        if  $contador[sol[i]] > 1$  then
          añadir  $i$  a  $robable$ 
        end if
      end for
       $robable \leftarrow \text{SHUFFLE}(robable)$ 
       $reparado \leftarrow$  falso
      for cada  $inst$  en  $robable$  do
         $old\_cluster \leftarrow sol[inst]$ 
         $\delta \leftarrow \text{CALCULATEINFEASIBILITYDELTA}(sol, inst, old\_cluster, c)$ 
        if  $\delta \leq 0$  then
           $contador[old\_cluster] \leftarrow contador[old\_cluster] - 1$ 
           $sol[inst] \leftarrow c$ 
           $contador[c] \leftarrow contador[c] + 1$ 
           $reparado \leftarrow$  verdadero
          break
        end if
      end for
      if  $\neg reparado$  y  $robable$  no vacía then
         $inst \leftarrow$  primera de  $robable$ 
         $contador[sol[inst]] \leftarrow contador[sol[inst]] - 1$ 
         $sol[inst] \leftarrow c$ 
         $contador[c] \leftarrow contador[c] + 1$ 
      end if
    end if
  end for
end function
```
